PROJECT REPORT ON DIGITAL LITERACY

Submitted by:

Name: Shorya Ranjan

Registration No: 25BCE10379

Course: B. Tech

Subject: Digital Literacy

Academic year: 2025-26

TASK 1

To create this visual guide for my fellow students, I turned to Canva, a popular and user-friendly digital design tool. It's perfect for quickly pulling together polished and clear infographics like this one, offering a vast library of icons and easy-to-manage layouts.

The infographic is structured as a single-page overview focusing on three pillars of digital citizenship. The top section in the green colour, defines digital literacy itself, breaking it down into key skills like finding quality information and creating ethical content. The middle orange section focuses on safety, listing actionable tips for securing accounts and avoiding scams. Finally, the purple section at the bottom covers professional email etiquette, which is so crucial for communicating with professors and future employers.

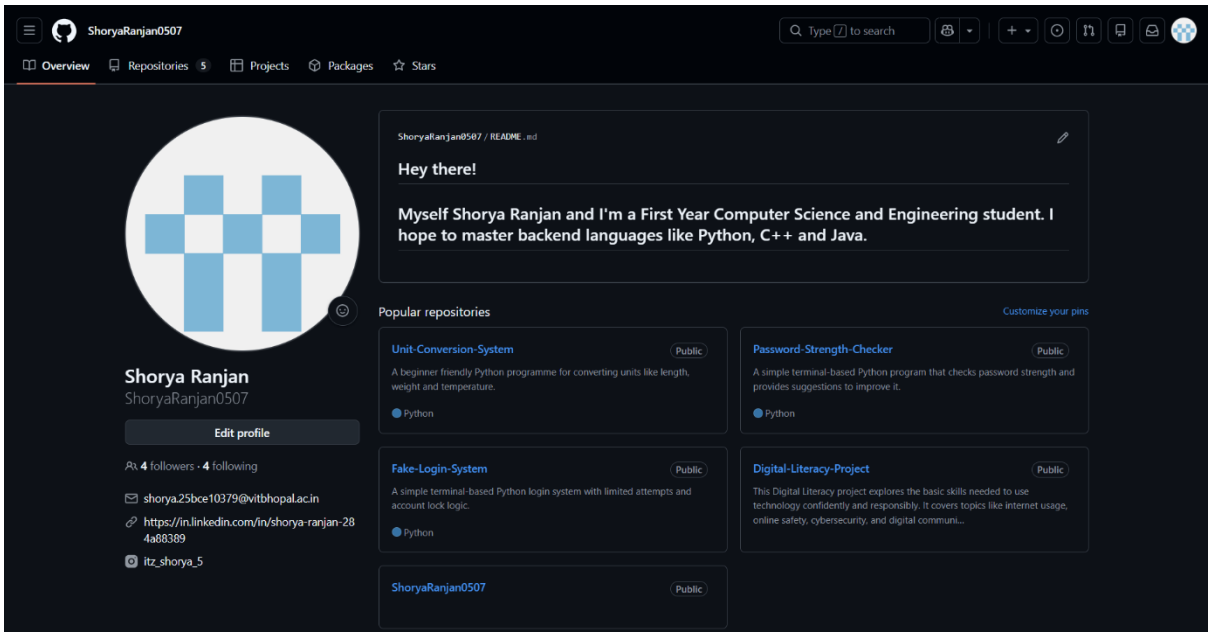
What I found most interesting while putting this together was the challenge of being minimalistic. My natural instinct is to include as much information as possible, but for this project, I had to resist that. The goal was to make it extremely easy to scan and digest, so I forced myself to shorten each concept down to a few very short, punchy points and use icons to do most of the talking.

TASK 2-

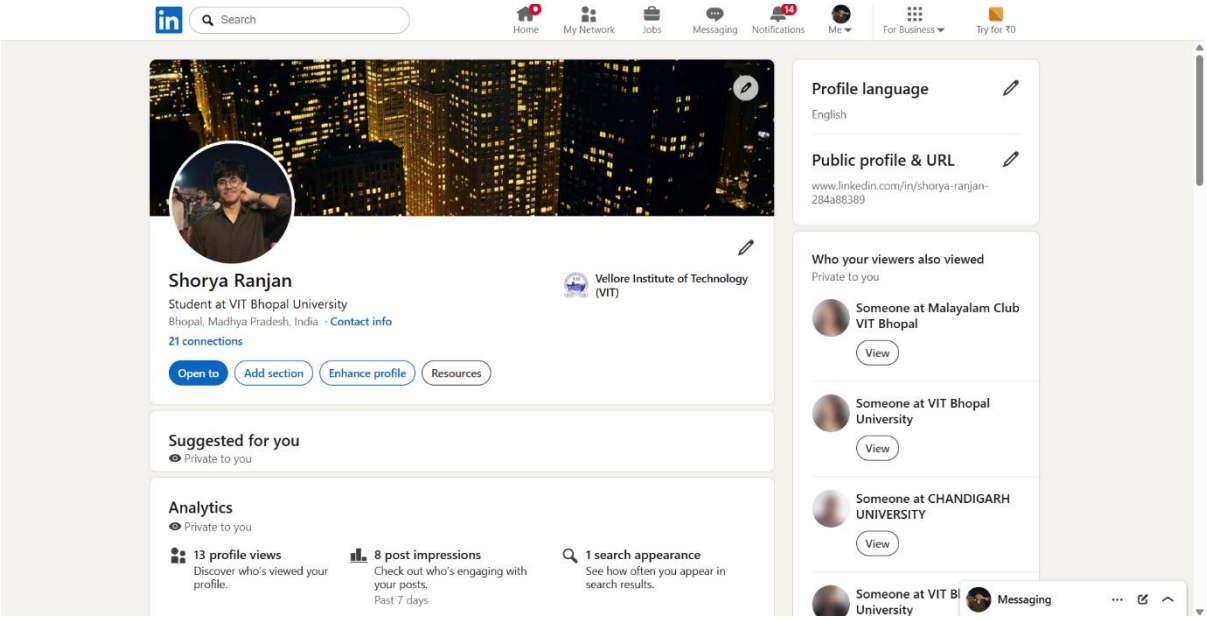
Building a professional online presence is a strategic move for any engineering student. I have selected **GitHub**, **LinkedIn**, and **Stack Overflow** as my core platforms to document my growth and connect with the global tech community over the next four years.

Platform Use

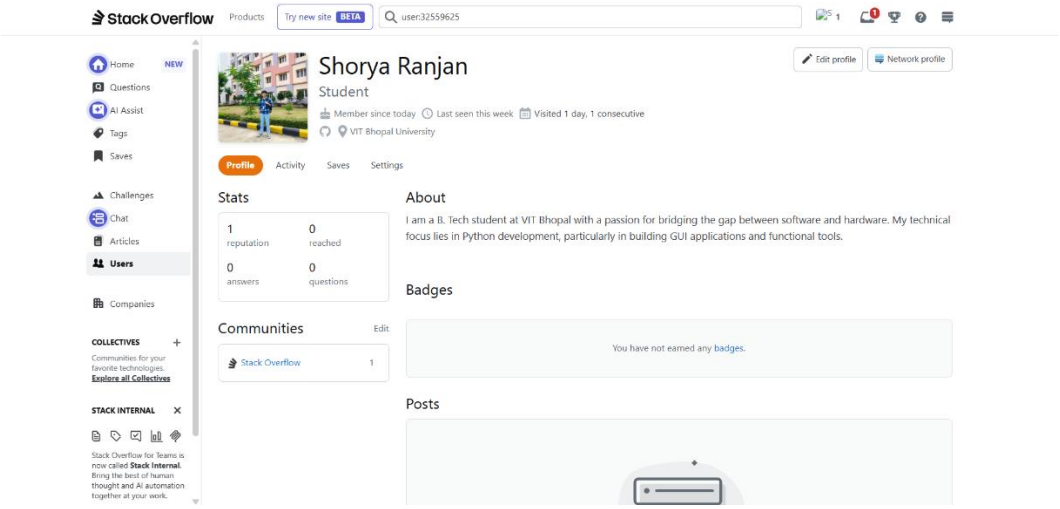
- **GitHub** serves as my open-source portfolio. It is where I host my Python projects, allowing others to review my code and version control skills.



- **LinkedIn** acts as my professional headquarters. It is essential for networking with industry experts, finding internships, and staying updated on corporate trends and job requirements.



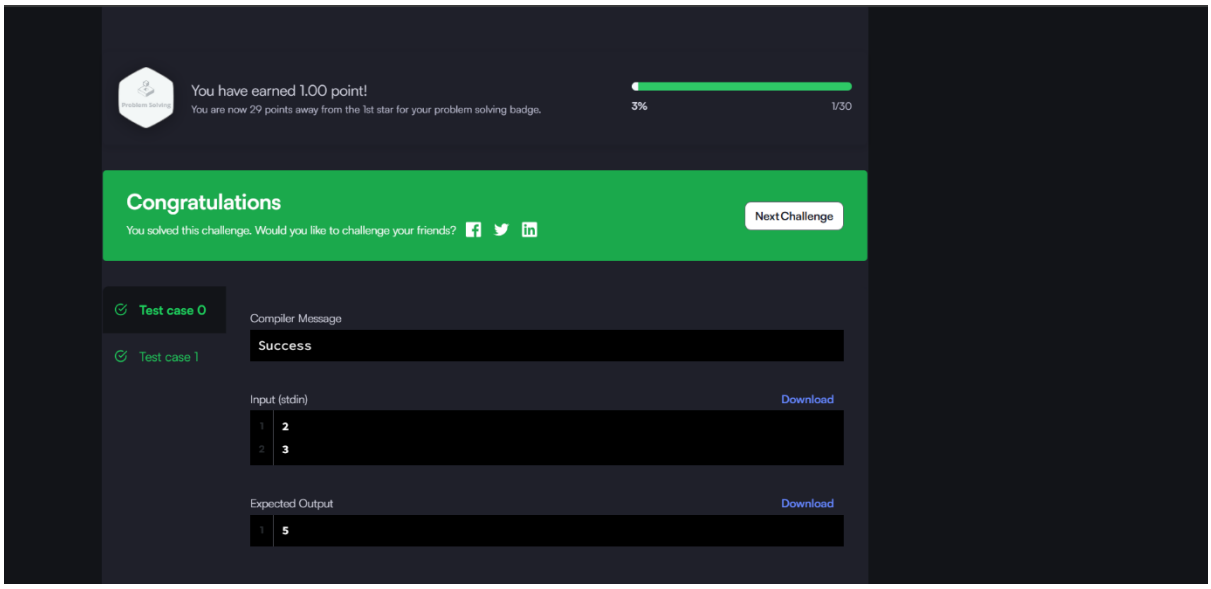
- **Stack Overflow** is my primary resource for collaborative problem-solving. It allows me to contribute to the developer community by asking high-quality questions and eventually providing solutions to complex technical hurdles.



Four-Year Plan

Throughout my degree, I plan to use these platforms to bridge the gap between software and hardware. On **GitHub**, I will transition from beginner scripts to complex and multi-layered repositories. On **LinkedIn**, I will document my journey from a student leader to a job-ready professional, sharing milestones and technical insights. Finally, I will aim to increase my reputation on **Stack Overflow**, moving from a learner to a contributor who helps fellow developers troubleshoot hardware-software integration. Together, these tools will provide a comprehensive record of my evolution as an engineer.

TASK 3-



Incorporating competitive programming into a technical routine is a vital step for any aspiring engineer. I have recently begun using **HackerRank** to sharpen my algorithmic thinking and coding efficiency, which complements my existing presence on platforms like GitHub and LinkedIn.

Platform Used

HackerRank feels like this focused spot for working on data structures and algorithms stuff i.e. DSA. Its not like those big general coding sites where you just dump code and hope. Here, you get instant feedback from all these automated tests that check if your code runs right and does not waste time or space. I am trying to tackle problems that start simple, like basic math logic, and go up to really tricky ones. That way, I figure I am strengthening my brain for those tech interviews and coding contests.

In school, tools like this make a big difference. They help me connect what I learn in my B Tech classes, things like logic gates or how complexity works in computing, to actual coding under pressure. It pushes me to use the ideas right away in timed challenges. I think practicing HackerRank a lot will boost how I handle lab exams, making me more sure and accurate. Over the four years ahead, sticking with it should link my regular lessons to the advanced skills for good software jobs. My logic base needs to match up with the hardware stuff I do hands on, or at least that is what it seems like now. Some parts of this might overlap more than I realize.

TASK 4-

One time I read about this big mess with Knight Capital back in 2012, it really stuck with me as a bad case of digital communication going wrong. A software glitch hit because they messed up a deployment, and in just 45 minutes they lost 440 million dollars, which is insane.

The problem started with some old dead code getting turned on by accident, plus the DevOps team and the deployment folks weren't talking clearly or documenting things well. Alerts came through that

were kind of vague, and the handbook for manual overrides wasn't spelled out enough, so engineers wasted time trying to figure out what was happening instead of just fixing it quick.

I think if they had better protocols in place, like standardized ways to communicate, a lot of that could have been avoided. Automated logs that show clear success or failure flags would help everyone know right away, and maybe a central channel for real-time incident stuff where the whole team sees the same info instantly. Peer reviews on digital checklists before deploying code might catch those little oversights too.

Clarity in digital stuff feels like it should be obvious, but this shows its not just nice to have, its what keeps things from falling apart financially and operationally. Sort of makes you wonder how often this happens without us hearing about it.

Task 5 -

What surprised me most during this research was the sheer sophistication of social engineering, the way scammers exploit human psychology rather than just technical glitches. Learning that a simple UPI Collect Request can be disguised as a "Refund" or "Prize" was a wake-up call; it highlights that even tech-enabled students can be vulnerable if they are distracted or rushed. It's a reminder that digital literacy isn't just about knowing how to code but about maintaining a zero-trust mindset in everyday interactions.

As a result, the one habit I am personally changing is my approach to multi-factor authentication. I'm moving away from SMS-based OTPs, which are susceptible to SIM-swapping, and switching to dedicated authenticator apps for my primary student and financial accounts. Security isnt something you just set up once and forget about. Its more like this constant thing you have to keep up with. That extra bit of hassle, you know, the friction they talk about, seems like no big deal when you consider how much safer it makes you feel overall. I mean, peace of mind counts for a lot, right? But yeah, its ongoing, not quick and done.